Guía Turística

Planificador de itinerarios para Guatemala

Documentación del proyecto y justificación de las tecnologías

Resumen

Guía Turística es una aplicación web pensada para usarse desde el teléfono que arma un itinerario de viaje por Guatemala a partir de lo que el usuario quiere hacer. El usuario indica el destino, las fechas, con quién viaja, su presupuesto, cómo se va a mover y qué le interesa; la aplicación busca lugares reales cercanos, revisa sus horarios, el clima pronosticado y los tiempos de traslado por calle, y propone un plan día por día con el orden de visita optimizado.

Durante el viaje la aplicación acompaña al usuario: muestra su posición en el mapa, lo guía con indicaciones paso a paso hasta la siguiente parada (recalculando si se desvía), avisa si se avecina lluvia y propone alternativas bajo techo, sugiere hoteles y restaurantes según el presupuesto y permite ajustar el plan conversando con un asistente.

Dato	Valor
Tipo de aplicación	Aplicación web progresiva (PWA) instalable en el teléfono
Alcance geográfico	Guatemala
Lenguajes	TypeScript (frontend y backend) y Python (optimizador de rutas)
Servicios propios	Frontend, backend (API), optimizador, PostgreSQL, Redis y Valhalla, levantados con Docker Compose
Forma de trabajo	Equipo con Git y GitHub: una rama por tarea, pull requests revisados y tickets GUI-xxx

1. Qué hace la aplicación

El flujo principal tiene cuatro pasos, uno por pantalla, para que en el teléfono cada decisión sea sencilla:

Paso	Qué hace el usuario	Qué hace el sistema
1. Destino	Elige el destino, las fechas, el tipo de viaje y sus intereses. Puede abrir un plan guardado.	Geocodifica el destino (solo dentro de Guatemala) y recupera los planes del usuario si inició sesión.
2. Hospedaje	Indica si ya tiene dónde quedarse o si necesita hotel.	Decide si más adelante se recomiendan hoteles y desde dónde parte cada día.
3. Contexto	Indica presupuesto y medio de transporte (a pie, auto, moto, transporte público…).	Prepara los parámetros con que se calcularán traslados y costos.
4. Itinerario	Ve su plan por días, el mapa, dónde dormir y comer, y la sección «En ruta».	Busca lugares, valida horarios, prioriza según el clima, optimiza el orden y genera el plan con IA.

Funciones principales

- **Generación del itinerario.** Se buscan lugares reales cerca del destino según los intereses, se descartan los que estarían cerrados a la hora de llegada y la IA arma los días. Si la IA no está disponible, un planificador sin IA genera un plan de respaldo.
- **Optimización del recorrido.** El orden de visita se calcula con tiempos reales por calle y respetando las ventanas de horario de cada lugar.
- **Clima.** Pronóstico por día y clima actual; los lugares se clasifican como bajo techo o al aire libre. Si durante el día aparece lluvia o tormenta, se proponen alternativas techadas y un nuevo recorrido que el usuario acepta o rechaza.
- **Hoteles y restaurantes.** Hoteles cercanos con su precio convertido a quetzales, filtrados para no pasar del 40 % del presupuesto diario.
- **Navegación guiada.** Ruta por calles hasta la siguiente parada con indicaciones en español; la línea se va borrando a medida que se avanza y la ruta se recalcula si el usuario toma otro camino.
- **Lugares abiertos ahora.** Lugares cercanos cuyo horario indica que están abiertos en este momento.
- **Asistente conversacional.** El usuario pide cambios («quita el museo», «quédate más tiempo en el lago») y el asistente propone ajustes que se aplican solo si los acepta.
- **Cuentas y planes guardados.** Registro e inicio de sesión, planes guardados y posibilidad de compartir un plan con un código.

2. Arquitectura

El sistema está separado en servicios con una responsabilidad clara cada uno. El frontend solo habla con el backend (salvo el mapa y las rutas de navegación, que van directo a Mapbox); el backend coordina las APIs externas, la base de datos, la caché y el optimizador.



Componente	Responsabilidad
Frontend	Pantallas del flujo, mapa, posición en vivo y navegación. Se instala como app en el teléfono.
Backend (API)	Autenticación, generación y validación de itinerarios, consultas a APIs externas, IA y persistencia.
PostgreSQL	Usuarios, viajes, itinerarios por día, paradas y sus restricciones climáticas.
Redis	Caché de respuestas de APIs externas (lugares, búsquedas) para ahorrar cuota y responder más rápido.
Optimizador	Microservicio que calcula el mejor orden de visita respetando horarios (problema de rutas con ventanas de tiempo).
Valhalla	Motor de rutas propio con el mapa de Guatemala: tiempos y distancias reales entre paradas, en auto o a pie.
APIs externas	Geoapify y Foursquare (lugares y horarios), OpenWeather (clima), SerpApi (hoteles), ExchangeRate-API (tipo de cambio), Gemini (IA) y Mapbox (mapa y navegación).

Los tipos de datos que comparten el frontend y el backend (viaje, parada, itinerario…) viven en una carpeta shared, de modo que si cambia la forma de un dato, ambos lados lo notan al compilar.

3. Por qué se eligió cada tecnología

Para cada pieza se indica qué alternativas había y por qué se tomó la decisión. Un criterio general guió todas: el proyecto es académico y de un equipo pequeño, así que se priorizaron herramientas **gratuitas o con capa gratuita**, que **corran igual en todas las computadoras** del equipo (vía Docker) y que permitan usar **un solo lenguaje** en la mayor parte del código.

Base de datos

PostgreSQL

Alternativas consideradas: SQL Server, MySQL, MongoDB

- **Gratis y de código abierto, sin ediciones ni licencias.** SQL Server tiene una edición gratuita (Express), pero con límites de tamaño y memoria y con la duda de licencias si el proyecto se publica; PostgreSQL no tiene ninguna de esas restricciones.
- **Corre de forma ligera en Docker.** La imagen oficial de PostgreSQL arranca en segundos y usa poca memoria. SQL Server para Linux pide al menos 2 GB de RAM solo para él, algo pesado si en la misma máquina corren también Valhalla, Redis y el optimizador.
- **Tipos que el modelo necesita.** Los intereses de un viaje se guardan como una lista de textos (*String[]*), que PostgreSQL soporta de forma nativa como arreglo; en SQL Server o MySQL habría que crear una tabla aparte o guardarlos como texto.
- **Preparado para datos geográficos.** El contenedor usa la imagen con **PostGIS**, la extensión de PostgreSQL más usada para consultas espaciales (lugares a cierta distancia, dentro de un área). Hoy las coordenadas se guardan como latitud y longitud, pero la puerta queda abierta sin cambiar de motor.
- **Prisma lo soporta de forma completa** y es de los motores más comunes en el ecosistema de Node.js, por lo que hay más documentación y ejemplos.
- **¿Por qué no MongoDB?** Los datos son claramente relacionales (un usuario tiene viajes, un viaje tiene días, un día tiene paradas, una parada tiene su restricción) y se necesitan relaciones con borrado en cascada y claves únicas; una base relacional encaja mejor.

Prisma (ORM)

Alternativas consideradas: TypeORM, Sequelize, SQL escrito a mano

- El esquema se escribe en un solo archivo legible y Prisma genera las **migraciones**: todo el equipo aplica los mismos cambios a su base con un comando, en orden, sin scripts sueltos.
- Genera un cliente **con tipos de TypeScript**: si una consulta pide un campo que no existe, el error aparece al compilar y no en producción.
- Evita la inyección SQL porque las consultas se construyen con parámetros.

Redis (caché)

Alternativas consideradas: Caché en memoria del propio servidor, guardar en PostgreSQL

- Las APIs de lugares y hoteles tienen **cuotas gratuitas limitadas**; guardar sus respuestas (24 horas para lugares, 15 minutos para búsquedas de hotel) evita repetir consultas idénticas.
- A diferencia de una caché en memoria, **sobrevive a reinicios** del backend y la expiración de cada dato es automática.
- Es mucho más rápido que guardar esas respuestas en PostgreSQL y no ensucia la base con datos temporales.

Backend

Node.js + TypeScript — Alternativas consideradas: Java (Spring), C# (.NET), Python (Django), JavaScript sin tipos

- **El mismo lenguaje que el frontend**: el equipo no tiene que cambiar de contexto y los tipos se comparten entre ambos lados.
- El trabajo del backend es sobre todo esperar respuestas de APIs externas; el modelo asíncrono de Node.js maneja muchas consultas en paralelo con poco consumo.
- TypeScript detecta errores de forma de datos antes de ejecutar, algo valioso con tantas APIs externas con respuestas distintas.

Fastify — Alternativas consideradas: Express, NestJS

- Es de los servidores más rápidos para Node.js y tiene **soporte nativo de TypeScript** y validación de entradas.
- Ofrece complementos oficiales para lo que se necesitaba (JWT para sesiones, CORS) sin tanta configuración como NestJS, que para un equipo pequeño añade mucha estructura.
- Express es más antiguo y no trae validación ni tipos de serie.

Zod, JWT y bcrypt — Alternativas consideradas: Validación manual, sesiones en servidor

- **Zod** valida todo lo que entra a la API y también las respuestas de la IA, que pueden venir con errores de formato.
- **JWT** permite sesiones sin guardar estado en el servidor, ideal para una app que se usa desde el teléfono.
- **bcrypt** guarda las contraseñas como hash; nunca se almacenan en texto plano.

Optimización de rutas

Python + FastAPI + OR-Tools — Alternativas consideradas: Implementar el algoritmo en Node.js, heurística propia

- Ordenar paradas respetando horarios de apertura es un **problema de rutas con ventanas de tiempo** (VRPTW), difícil de resolver bien a mano. **Google OR-Tools** es una librería libre y de referencia para este tipo de problema.
- OR-Tools tiene su mejor soporte y documentación en Python, por eso el optimizador es un **microservicio aparte** en ese lenguaje.
- **FastAPI** permite exponerlo como una API pequeña, con validación automática y documentación generada, que el backend consume por HTTP.

Valhalla — Alternativas consideradas: Google Distance Matrix, OSRM, línea recta

- Primero se usó la distancia en línea recta con una velocidad fija, lo cual no refleja las calles reales. Valhalla calcula tiempos y distancias **por la red vial**.
- Es **de código abierto y corre en nuestro propio contenedor** con el mapa de Guatemala de OpenStreetMap: no hay costo por consulta, a diferencia de Google, que cobra cada elemento de la matriz de distancias.

- Con un solo grafo sirve para **auto y a pie**; OSRM, la otra opción libre, necesita preparar un grafo distinto por cada modo de transporte.

Frontend

React + Vite

Alternativas consideradas: Angular, Vue, Next.js

- React es la librería más conocida por el equipo y la que tiene más componentes y ayuda disponibles.
- La app es una sola página que no necesita posicionarse en buscadores, así que no hace falta el renderizado en servidor de Next.js.
- **Vite** arranca al instante y recarga los cambios en caliente, lo que agiliza el desarrollo.

Aplicación web progresiva (PWA)

Alternativas consideradas: App nativa (Android/iOS), React Native

- El turista usa la app en la calle, desde el teléfono. Como PWA se **instala desde el navegador**, tiene ícono propio y guarda archivos para cargar rápido, sin publicar en tiendas ni mantener dos aplicaciones nativas.
- Da acceso al GPS, que es lo que necesita la navegación.

Mapbox GL

Alternativas consideradas: Google Maps, Leaflet + OpenStreetMap

- Dibuja mapas vectoriales fluidos en el teléfono y permite capas propias: la ruta del día, el tramo de navegación, los marcadores.
- Su API de direcciones devuelve la ruta por calles con **indicaciones paso a paso en español**, base de la navegación guiada.
- Tiene una capa gratuita amplia; Leaflet es libre pero no trae navegación, y Google Maps tiene costos más altos por uso.

Inteligencia artificial y datos externos

Gemini (Google)

Alternativas consideradas: OpenAI (GPT), Claude, sin IA

- Ofrece una **capa gratuita** suficiente para un proyecto académico y un SDK oficial para Node.js.
- Puede devolver **JSON estructurado**, que luego se valida con Zod antes de usarlo.
- La IA no es indispensable: si falla o no hay clave, un **planificador sin IA** arma el itinerario igual, y el usuario recibe un aviso claro.

Servicio	Para qué se usa	Por qué este
Geoapify	Lugares cercanos por categoría y sus horarios	Capa gratuita generosa, datos de OpenStreetMap, horarios en formato estándar
Foursquare	Segunda fuente de lugares	Amplía la variedad de lugares cuando Geoapify devuelve pocos resultados
OpenWeather	Pronóstico y clima actual	Estándar de facto, gratis para el volumen del proyecto

Servicio	Para qué se usa	Por qué este
SerpApi	Hoteles (Google Hotels) con precio	Acceso a precios reales de Google Hotels sin integrarse con cada cadena
ExchangeRate-API	Conversión de USD a quetzales	Gratuita y simple; los precios de hoteles vienen en dólares

Infraestructura y calidad

Docker Compose

Alternativas consideradas: Instalar cada servicio a mano

- Con un comando cada integrante levanta PostgreSQL, Redis, el optimizador y Valhalla **con las mismas versiones**, sin el clásico «en mi máquina sí funciona».

Pruebas automáticas

Alternativas consideradas: Jest, Vitest, sin pruebas

- El backend y el frontend usan el **runner de pruebas integrado de Node.js** (sin dependencias extra); el optimizador usa **pytest**.
- Las pruebas cubren lo que más se rompe: validación de horarios, cálculo de rutas y navegación, clima, hoteles y conversión de datos de las APIs.

Git + GitHub

Alternativas consideradas: —

- Cada tarea tiene un ticket (GUI-xxx), una rama propia y un pull request que se revisa antes de unirse a la rama principal.

4. Modelo de datos

La base de datos es pequeña y relacional. Cada viaje puede pertenecer a un usuario (o ser anónimo) y se divide en días:

Tabla	Contenido	Relación
Usuario	Correo (único), contraseña con hash bcrypt, nombre	Tiene muchos viajes
Viaje	Destino y coordenadas, fechas, tipo de viaje, presupuesto, transporte, intereses, hospedaje, código para compartir	Pertenece a un usuario; tiene un itinerario por día
Itinerario	Número de día y fecha	Pertenece a un viaje; tiene muchas paradas
Parada	Lugar, coordenadas, horario, tiempo de estancia y orden de visita	Pertenece a un día; puede tener una restricción
Restricción	Si el lugar es bajo techo o al aire libre y su sensibilidad al clima	Una por parada

Al borrar un viaje se borran en cascada sus días, paradas y restricciones, algo que la base garantiza por sí sola.

5. Limitaciones y trabajo futuro

- El alcance es Guatemala: la geocodificación, el mapa de Valhalla y la moneda están pensados para el país.
- El optimizador integra dos modos de transporte (auto y a pie); moto, transporte público y aéreo se calculan como auto.
- Los precios de hoteles son referenciales y pueden diferir del cargo final.
- Las consultas de cercanía se hacen en las APIs externas; PostGIS ya está disponible para hacerlas en la propia base si el volumen lo requiere.
- Los horarios dependen de lo que publican Geoapify y Foursquare; un lugar sin horario verificable se trata con cautela o se excluye.